

Chapter 7

Generative AI

For much of its history, computer vision focused primarily on perceptual and analytical tasks: identifying objects, segmenting regions, and mapping high-dimensional visual inputs into low-dimensional semantic labels. Generative Artificial Intelligence shifts the paradigm from analysis to synthesis. Rather than asking how to extract information from an image, generative modeling addresses a more fundamental probabilistic challenge: how to learn, approximate, or sample from the true underlying distribution $p(x)$ of visual data.

Modeling $p(x)$ over high-dimensional pixel grids requires balancing expressive capacity, computational tractability, and sample quality. To navigate these trade-offs, several distinct architectural paradigms have emerged, such as autoregressive models, (variational) autoencoders and generative adversarial networks.

This chapter provides a unified theoretical and practical foundation for generative modeling in computer vision. We examine the trade-offs between exact likelihood estimation, latent space structure, sampling efficiency, and training stability across autoregressive, variational, and adversarial frameworks.

7.1 Model Paradigms

A central distinction in machine learning—and one that becomes especially important in computer vision—is whether a model predicts a target directly from an input or learns a distribution from which data can be generated. Understanding this distinction clarifies what each type of model can and cannot do.

7.1.1 Discriminative Models

A **discriminative model** learns a direct mapping from an input x to a target y . Probabilistic discriminative models learn the conditional distribution

$$p(y \mid x),$$

where x is an input (e.g. an image) and y is a label or target (e.g. the class of the image). Other discriminative models, such as support vector machines, learn a decision function rather than a probability. Logistic regression, support vector machines, and most modern deep convolutional classifiers belong to this category.

Note

The decision function can be “transformed” into a probability using calibration.

Because discriminative models only need to capture the relationship between x and y , they tend to achieve high predictive accuracy on the tasks for which they are trained. However, they do not by themselves model $p(x)$, so they cannot directly generate new images or evaluate the likelihood of an input. They may still be used as part of an out-of-distribution detection system, but they do not automatically provide a model of the input data.

7.1.2 Generative Models

A **generative model** learns a distribution over the data. In the unconditional setting, it models the marginal distribution $p(x)$. If labels are included, it may instead learn the joint distribution

$$p(x, y) = p(x | y) p(y),$$

or the class-conditional distribution $p(x | y)$. When $p(x | y)$ and the class prior $p(y)$ can be evaluated, Bayes’ theorem gives

$$p(y | x) = \frac{p(x | y) p(y)}{\sum_{y'} p(x | y') p(y')}.$$

Thus, a class-conditional generative model can also be used for classification, although it has to model more information than a discriminative classifier.

The defining capability of generative models is *synthesis*: from a trained model, one can draw novel samples $\tilde{x} \sim p(x)$ that resemble the training data. Some generative models provide an **exact likelihood**, such as PixelCNN, some provide an **approximation**, such as VAEs, and others only provide a **sampling procedure**, such as GANs. In computer vision, these models enable image generation, data augmentation, inpainting, and other image-editing tasks.

7.1.3 Conditional Models

A **conditional generative model** is a generative model that conditions synthesis on an auxiliary variable c :

$$p(x | c).$$

The conditioning signal c can take many forms: a class label, a text prompt, a low-resolution image, a semantic segmentation map, or a depth channel. Conditional models retain the ability to synthesise data while giving the practitioner explicit control over the output.

Conditional and unconditional generation are therefore two forms of generative modelling, rather than separate model families. Depending on the architecture, c may be concatenated with the input, projected into intermediate layers, used for conditional normalisation, or supplied through cross-attention. Conditional image synthesis is used in text-to-image generation, super-resolution, image-to-image translation, and controllable video synthesis.

7.2 Autoregressive Models

Autoregressive models provide a principled approach to density estimation by decomposing the joint distribution of an image into a product of conditional distributions. When each conditional distribution is normalised, they provide an exact, tractable likelihood.

7.2.1 Tractable Density via the Chain Rule

Consider an image with $N = HW$ spatial positions, written in **raster-scan** order (left-to-right and top-to-bottom) as $x = (x_1, \dots, x_N)$. For a grayscale image, x_i is one intensity; for an RGB image, $x_i = (x_{i,R}, x_{i,G}, x_{i,B})$ is one pixel vector. The probability chain rule gives

$$p(x) = \prod_{i=1}^N p(x_i \mid x_1, \dots, x_{i-1}) = \prod_{i=1}^N p(x_i \mid x_{<i}),$$

where $x_{<i} = (x_1, \dots, x_{i-1})$. For 8-bit images, each grayscale intensity or RGB channel value belongs to $\{0, \dots, 255\}$. The RGB channels inside each pixel will be factorised separately below.

The parameters are learned by maximising the log-likelihood over a training set $\mathcal{D}$,

$$\max_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \prod_{i=1}^N p_{\theta}(x_i \mid x_{<i}) = \max_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \log \left(\prod_{i=1}^N p_{\theta}(x_i \mid x_{<i}) \right) = \max_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \sum_{i=1}^N \log(p_{\theta}(x_i \mid x_{<i})).$$

Equivalently, training minimises the **negative log-likelihood (NLL)**, which is a sum of cross-entropy terms for the observed pixel values.

Example: Raster-scan factorisation

Consider a 2×2 grayscale image whose pixels are ordered as

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}.$$

Its joint distribution is factorised as

$$p(a, b, c, d) = p(a) p(b \mid a) p(c \mid a, b) p(d \mid a, b, c).$$

Each new pixel is therefore predicted from all earlier pixels in raster order.

7.2.2 PixelCNN

PixelCNN parameterises each conditional distribution with a deep convolutional network that operates over the entire image but is constrained to access only the *previously seen* values at each position. This constraint is enforced through **masked convolutions**.

Masked Convolutions

Recall that a standard $k \times k$ convolution at position (i, j) aggregates information from a symmetric window centred on (i, j) . In an autoregressive model, the prediction at (i, j) must not depend on pixels that come after it in raster order. The solution is to multiply the kernel weights element-wise by a fixed binary mask M that removes all forbidden connections. The kernel weights are learned normally, while the mask is not trainable.

The raster-scan order allows the prediction at (i, j) to use all pixels in earlier rows and all pixels to its left in the same row. Whether the centre of the kernel is included depends on the type of mask:

- **Mask A:** It is used in the first convolutional layer, whose input contains the raw pixels. For a grayscale image, Mask A excludes the centre of the kernel, ensuring that the model cannot see the value it is predicting.
- **Mask B:** It is used in subsequent layers, whose inputs are hidden feature maps. It includes the centre of the kernel. This is safe because the hidden representation at the centre was already constructed using only valid past context.

For a $k \times k$ kernel with $k = 2r + 1$, let (m, n) denote a kernel position and (r, r) its centre. The grayscale spatial masks are

$$M_{m,n}^A = \begin{cases} 1 & \text{if } m < r \text{ or } (m = r \text{ and } n < r), \\ 0 & \text{otherwise,} \end{cases}$$

$$M_{m,n}^B = \begin{cases} 1 & \text{if } m < r \text{ or } (m = r \text{ and } n \leq r), \\ 0 & \text{otherwise.} \end{cases}$$

The masked weights are $\widetilde{W} = W \odot M$, where $\odot$ denotes element-wise multiplication. This is shown visually in Figure 7.1.

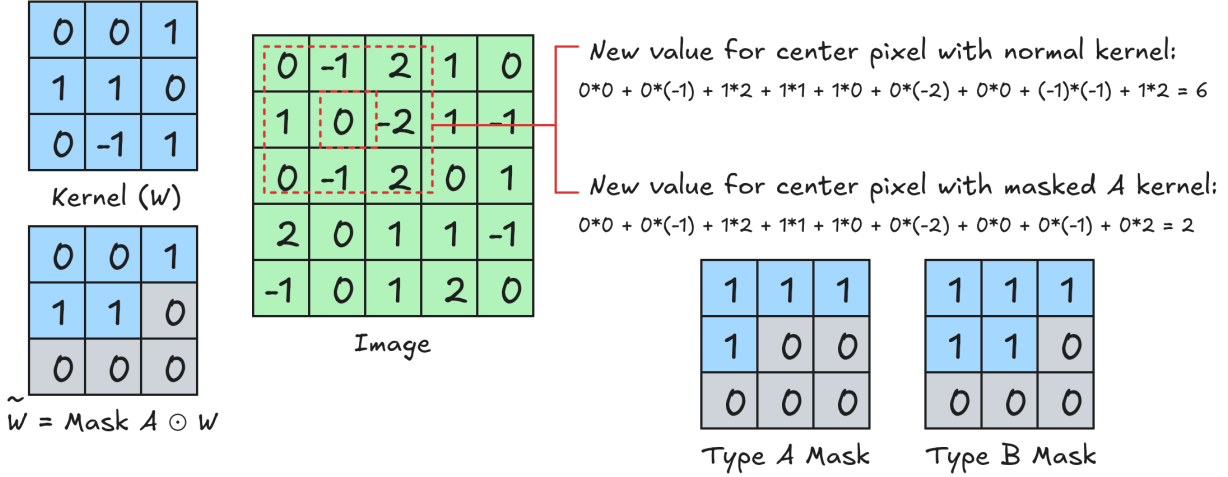


Figure 7.1: Example of masked convolution.

RGB Channel Ordering

For an RGB pixel $x_i = (x_i^R, x_i^G, x_i^B)$, the channels are also generated autoregressively. Red is predicted first, then green using red, and finally blue using red and green:

$$p(x_i | x_{<i}) = p(x_i^R | x_{<i}) p(x_i^G | x_{<i}, x_i^R) p(x_i^B | x_{<i}, x_i^R, x_i^G).$$

This ordering changes only the centre of the spatial mask. With rows denoting output groups and columns denoting input groups, both ordered as (R, G, B) , the centre masks are

$$M_{\text{centre}}^A = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}, \quad M_{\text{centre}}^B = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}.$$

Thus, Mask A prevents a channel from seeing itself but allows it to use earlier channels of the same pixel. Mask B also allows each channel group to use its own hidden features. At valid off-centre positions, all channel groups may be used.

These 3×3 matrices are a conceptual illustration. A practical PixelCNN has many hidden feature maps divided into R , G , and B groups. The mask compares these groups, not arbitrary hidden-channel indices.

Figure 7.2 visualises these centre connections. Each row represents an output group, each column represents an input group, and a value of 1 indicates an allowed connection.

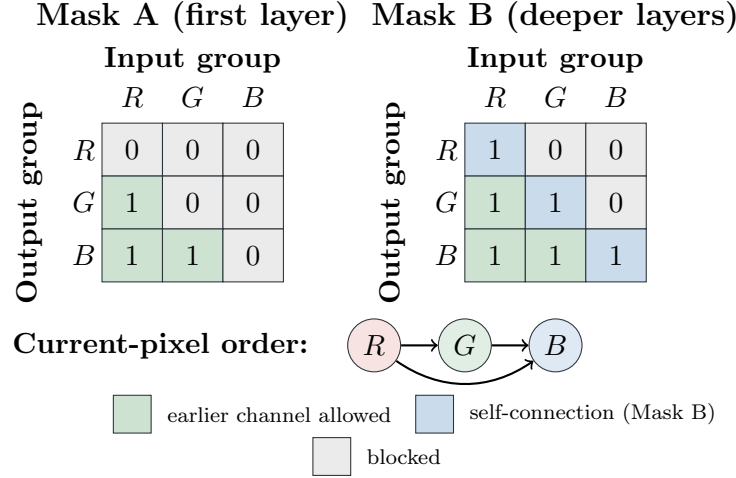


Figure 7.2: Centre-channel masks for an RGB PixelCNN. Rows denote output groups and columns denote input groups. Mask A uses only earlier channels of the current pixel, while Mask B also permits self-connections in deeper layers.

Example: Mask A on a small RGB image

Consider the following 3×3 RGB patch, where the centre pixel is the one being predicted:

$$X = \begin{pmatrix} (1, 1, 1) & (2, 2, 2) & (3, 3, 3) \\ (4, 4, 4) & \boxed{(10, 20, 30)} & (6, 6, 6) \\ (7, 7, 7) & (8, 8, 8) & (9, 9, 9) \end{pmatrix}.$$

Suppose that every allowed kernel weight is 1. Schematically, the Mask A kernel is

$$\tilde{K} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & M_{\text{centre}}^A & 0 \\ 0 & 0 & 0 \end{pmatrix},$$

where each spatial entry represents a block of channel connections. At the current pixel, its contribution to the three output groups is

$$M_{\text{centre}}^A \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix} = \begin{pmatrix} 0 \\ 10 \\ 30 \end{pmatrix}.$$

Therefore, the red output uses no channel from the current pixel, the green output can use its red value (10), and the blue output can use its red and green values ($10 + 20 = 30$). The four earlier spatial pixels can contribute to all three output groups, while future pixels remain blocked.

Blind spot Although one masked kernel includes the complete rows above it, repeatedly stacking a single stream of masked convolutions can omit part of the valid upper-right context from the effective receptive field. This is shown in Figure 7.3, where the CNN consists of

three convolutions.

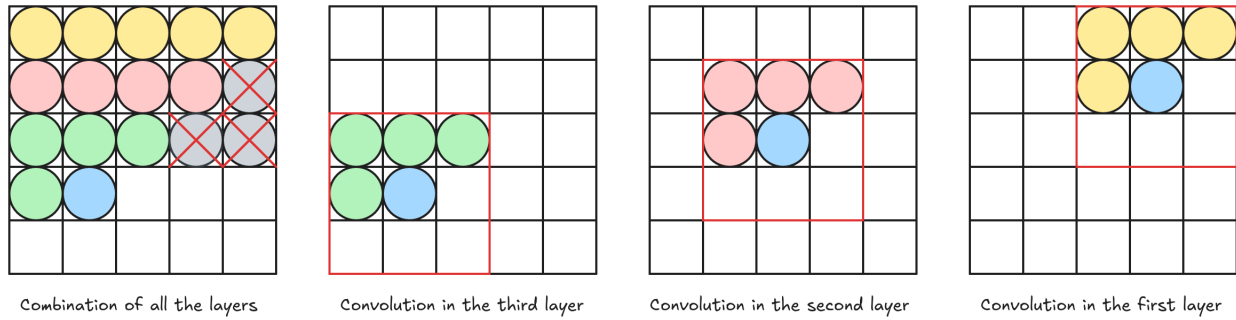


Figure 7.3: Blind spot: pixel (4, 2) should “see” pixels in positions (3, 4), (3, 5) and (2, 5). Here we assume indexes start at 1, not 0.

Architecture

The following describes a common original-style PixelCNN architecture. Other PixelCNN variants may use different blocks or output distributions. The model has three stages, also represented in Figure 7.4 along with the main tensor shapes:

- **Input layer:** The input image $x \in \{0, \dots, 255\}^{H \times W \times C}$ is first passed through a 7×7 Mask A convolution with F output feature maps. The relatively large kernel gives the network a broad initial context while the mask preserves the autoregressive order.
- **Residual blocks:** The bulk of the network consists of L residual blocks using Mask B. A typical bottleneck block follows this design:
 1. ReLU followed by a masked 1×1 convolution that reduces the number of feature maps from F to $F/2$.
 2. ReLU followed by a 3×3 Mask B convolution.
 3. ReLU followed by a masked 1×1 convolution that restores the number of feature maps to F .
 4. Element-wise addition of the block input (residual connection).

The residual connection helps gradients flow to earlier layers. All convolutions, including the 1×1 convolutions (note they are just M_{centre}^B), preserve the RGB group ordering so that no future information is introduced.

- **Output head:** After the residual stack, masked 1×1 convolutions map the feature vector at each position to $256C$ logits. Hidden head layers may use ReLU, but the final logits must not have a ReLU because logits can take any real value. For a batch of B images, the output can be reshaped conceptually as $(B, H, W, C, 256)$, and softmax is applied over the final dimension.

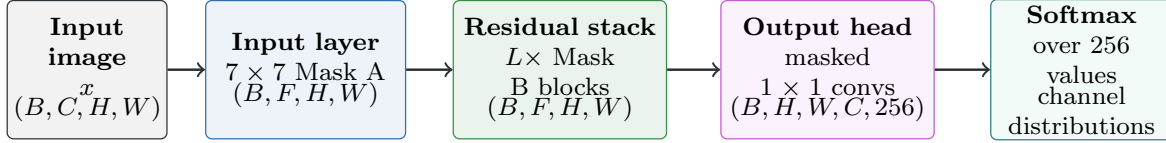


Figure 7.4: A common PixelCNN architecture. Mask A prevents access to the value being predicted, Mask B processes valid hidden context, and the output head produces 256 logits for every pixel and colour channel.

Output Distribution

For an 8-bit image, PixelCNN can model each colour-channel value with a categorical distribution over the 256 possible intensities. At raster position i and channel $c \in \{R, G, B\}$,

$$p_{\theta}(x_{i,c} \mid x_{<i}, x_{i,<c}) = \text{softmax}(f_{\theta,i,c}(x)).$$

This categorical head assumes discrete integer-valued images.

The network is trained end-to-end by minimising the negative log-likelihood, which decomposes into one cross-entropy term per pixel and channel over the training set:

$$\mathcal{L}_{\text{NLL}}(\theta) = - \sum_{\mathbf{x} \in \mathcal{D}} \sum_{i=1}^N \sum_{c \in \{R, G, B\}} \log(p_{\theta}(x_{i,c} \mid x_{<i}, x_{i,<c})).$$

Example: One channel's contribution to the NLL

Suppose the true label for a particular channel is 120 and the model assigns it probability 0.08. Its contribution to the NLL is

$$-\log p(x_{i,c} = 120) = -\log(0.08) \approx 2.53.$$

Inference: Image Generation

At inference time, there is no target image. We begin with an empty image canvas and generate its values in raster order. The initial values assigned to future pixels do not affect the current prediction because the masks block access to them.

For every pixel $i = 1, \dots, N$, the channels are generated in the order $R \rightarrow G \rightarrow B$:

1. Pass the partially generated image through PixelCNN.
2. Obtain the predicted distribution $\hat{p}_{i,c} \in \mathbb{R}^{256}$ for the current channel c .
3. Sample its intensity from that distribution:

$$x_{i,c} \sim \text{Categorical}(\hat{p}_{i,c}).$$

4. Insert the sampled value into the image and continue with the next channel or pixel.

After all *HWC* channel values have been sampled, the canvas contains the generated image. Unlike training, this process is sequential because each new prediction depends on values generated in earlier steps.

7.2.3 Strengths and Limitations

Strengths

- **Exact, tractable likelihood.** The chain-rule factorization gives a closed-form likelihood and permits density estimation. Likelihoods should only be compared when models use the same data representation and preprocessing. Results are often reported in bits per dimension.
- **Training stability.** Maximum-likelihood training uses a standard cross-entropy objective and is generally simpler than adversarial min-max optimisation.
- **Flexible output distributions.** Each conditional can use a categorical distribution or another suitable likelihood, allowing the model to represent multiple possible pixel values.

Limitations

- **Sequential sampling.** A straightforward RGB PixelCNN makes up to *HWC* sequential channel decisions, making generation slow and difficult to parallelise.
- **Blind spot.** The original single-stack architecture can omit part of the valid upper-right context.
- **No compact latent space.** A plain PixelCNN does not learn a compact continuous latent representation, which makes interpolation and semantic manipulation less direct.

7.3 AutoEncoders and Variational AutoEncoders

7.3.1 AutoEncoders (AEs)

A standard **autoencoder** is a neural network trained to reproduce its input. It has two main components:

- The **encoder** maps an input image x to a latent representation z : $z = g(x)$.
- The **decoder** maps z back to a reconstruction of the input: $\hat{x} = f(z)$.

The complete model is therefore $x \xrightarrow{\text{encoder}} z \xrightarrow{\text{decoder}} \hat{x}$.

The latent representation is usually smaller than the input, forcing the encoder to preserve the most useful information. For images, the encoder and decoder are commonly built using convolutional and upsampling layers, respectively, as shown in Figure 7.5.

The input image is also the target. For a batch of B images, a simple reconstruction loss is the mean squared error (MSE):

$$\mathcal{L}_{\text{rec}} = \frac{1}{B} \sum_{n=1}^B (x_n - \hat{x}_n)^2 = \frac{1}{B} \sum_{n=1}^B (x_n - f(g(x_n)))^2.$$

Note

Binary cross-entropy or other reconstruction losses may be used depending on the image representation, for example if we not only want to reconstruct the input but also to obtain the class it belongs to.

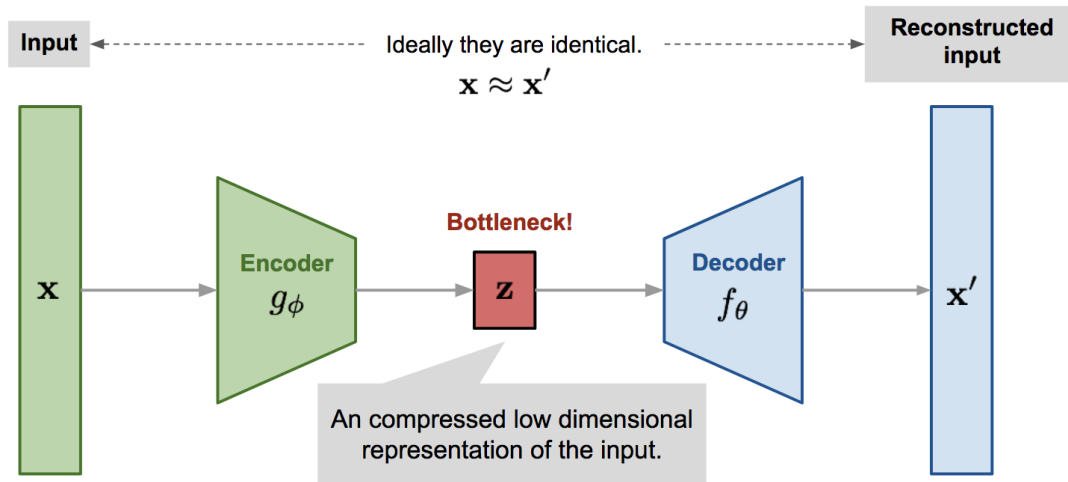


Figure 7.5: AutoEncoder schema.

After training, z can be useful for dimensionality reduction, feature extraction, retrieval, or anomaly detection. However, a standard autoencoder does not specify how latent vectors are distributed.

Warning: Image Generation with AEs

Sampling an arbitrary z and try to reconstruct it may produce a poor or meaningless image. For this reason, a standard AE is not by itself a complete generative model.

7.3.2 Variational AutoEncoders (VAEs)

A **Variational AutoEncoder** turns the latent space into a probabilistic one. Instead of mapping an image to a single latent vector, the encoder predicts the parameters of a distri-

bution. The usual choice is a diagonal Gaussian:

$$q(z | x) = \mathcal{N}(z; \mu(x), \text{diag}(\sigma^2(x))) .$$

Here, $\mu(x)$ and $\sigma(x)$ are vectors produced by the encoder g . A latent vector is obtained as

$$z = \mu(x) + \sigma(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

This is called the **reparameterization trick**. The randomness is placed in ε , allowing gradients to pass through μ and σ during training. If we just sampled x from a $N(\mu(x), \sigma(x))$ we could not perform backpropagation.

The decoder receives the sampled vector and reconstructs the image:

$$\hat{x} = f(z).$$

This whole process is shown in Figure 7.6.

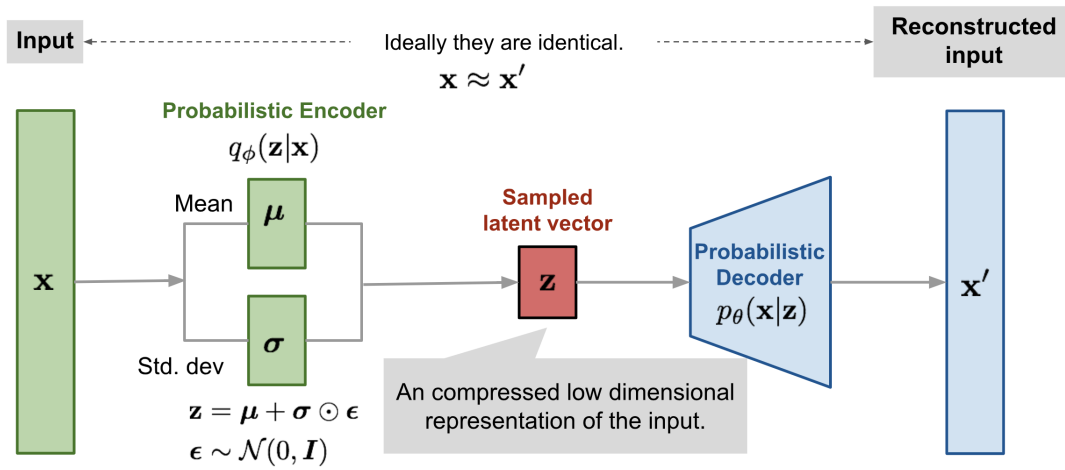


Figure 7.6: Variational AutoEncoder schema.

The VAE loss combines two objectives for the **loss**:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{rec}} + D_{\text{KL}}(q(z | x) \| \mathcal{N}(0, I)) ,$$

where $q(z | x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))) = g(x)$.

The reconstruction term encourages $\hat{x}$ to resemble x . The Kullback-Leibler (KL) term encourages the encoded distributions to remain close to the standard normal distribution. This makes the latent space more continuous and gives the model a known distribution from which to sample. Note that we could also include more terms to the loss, for example a classification term, as in the AutoEncoder.

At **inference** time, an input image is not required. We sample

$$z \sim \mathcal{N}(0, I)$$

and pass it directly through the decoder. Different samples of z produce different images. Unlike PixelCNN, the decoder normally generates all pixels in one forward pass.

7.3.3 AEs and VAEs Variants

Many variants change either the training objective or the latent representation. Four common examples are:

- **Denoising Autoencoder:** The input $\tilde{x} = x + \varepsilon$ is deliberately corrupted with noise, but the target x of the predicted image $\hat{x}$ remains the original clean image:

$$\tilde{x} \xrightarrow{\text{AE}} \hat{x} \approx x.$$

The model must learn useful image structure instead of simply copying its input. Denoising AEs can be used for image restoration and representation learning.

- **Sparse Autoencoder:** A sparsity penalty in the loss function encourages most latent activations to be zero or close to zero. As a result, only a small number of latent units are active for a given image. This can produce more selective features even when the latent vector is not very small.
- **β -VAE:** A β -VAE changes the weight of the KL term:

$$\mathcal{L}_{\beta\text{-VAE}} = \mathcal{L}_{\text{rec}} + \beta D_{\text{KL}}(q(z | x) \| \mathcal{N}(0, I)).$$

Values $\beta > 1$ encourage a more strongly organised latent space, but may reduce reconstruction quality. The parameter therefore controls the trade-off between reconstruction and latent regularisation.

- **Vector-Quantised VAE (VQ-VAE):** A VQ-VAE uses a learned collection of latent vectors called a **codebook**. Each pixel of the encoder output is replaced by its nearest codebook value before being passed to the decoder. The latent representation is therefore discrete rather than continuous. This is useful for representing an image as a grid of visual tokens that can later be modelled by another generative model. Because the arg min part of the quantisation (replacing with nearest codebook vector) is non-differentiable, we must use the gradients from the decoder $\nabla_z L$ for the encoder output backpropagation (line red). A representation is shown in Figure 7.7.

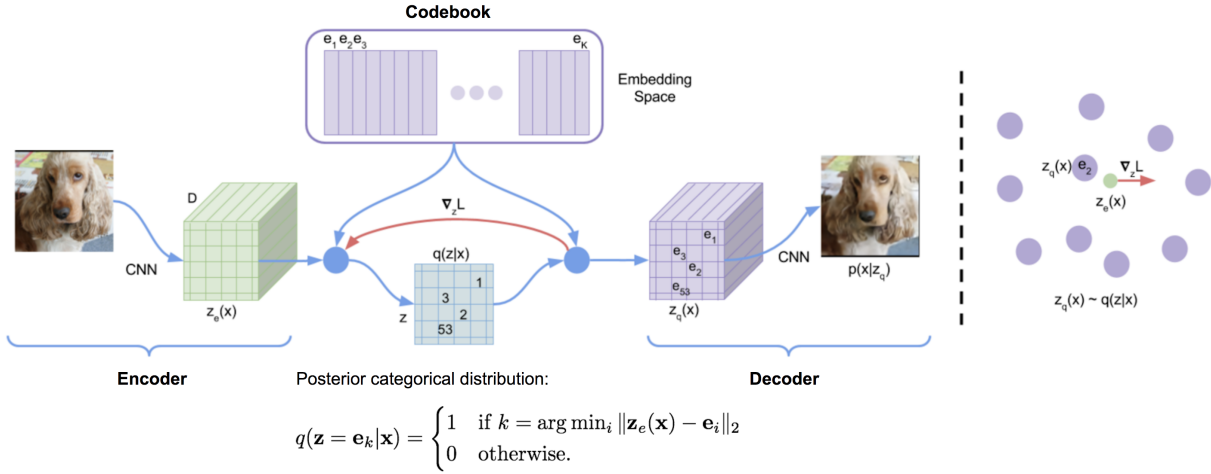


Figure 7.7: VQ-VAE schema.

7.4 Generative Adversarial Networks (GANs)

7.4.1 Idea

A **Generative Adversarial Network** contains two neural networks that are trained together:

- The **generator** G transforms a random latent vector z into a synthetic image:

$$z \sim p_z, \quad \hat{x} = G(z).$$

A common choice is $p_z = \mathcal{N}(0, I)$. We can start from $z \in \mathbb{R}^{c \times h \times w}$ and apply some convolutions in such a way that we finish with the desired dimensions.

- The **discriminator** D receives an image x and predicts the probability that it is real:

$$D(x) \in [0, 1].$$

Real training images have target 1, while generated images have target 0. We can apply some convolutions to $G(z)$, flatten the vector, and use a MLP to obtain the final classification.

The generator and discriminator have opposing objectives. The discriminator tries to distinguish real images from generated ones, while the generator tries to create images that the discriminator classifies as real. This process is illustrated in Figure 7.8.

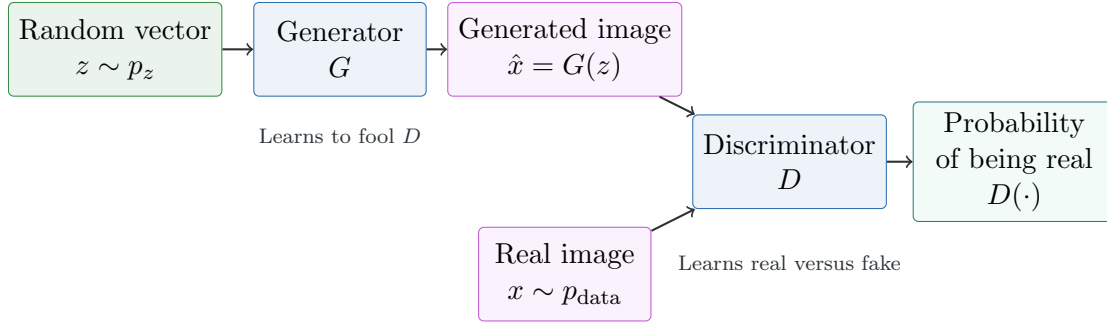


Figure 7.8: GAN training. The generator converts random latent vectors into images, while the discriminator learns to distinguish generated images from real training images.

Adversarial Losses

For a batch containing B real images x_n and B latent samples z_n , the discriminator minimises the binary cross-entropy loss

$$\mathcal{L}_D = -\frac{1}{B} \sum_{n=1}^B [\log(D(x_n)) + \log(1 - D(G(z_n)))].$$

The first term rewards high predictions for real images. The second rewards low predictions for generated images.

The generator is trained using the opposite target: it wants generated images to be classified as real. A commonly used generator loss is

$$\mathcal{L}_G = -\frac{1}{B} \sum_{n=1}^B \log(D(G(z_n))).$$

Therefore, the generator receives no target image to reconstruct. It learns from the discriminator's feedback.

Alternating Training

The two networks are updated alternately:

1. Sample a batch of real images and latent vectors.
2. Generate fake images $G(z)$.
3. Update D to classify real images as 1 and generated images as 0.
4. Sometimes steps 2 and 3 can be repeated k times, in such a way that the number of times we update discriminator's parameters vs generator's parameters is $k : 1$.
5. Recompute $G(z)$ and update G so that $D(G(z))$ becomes close to 1. The parameters of D are frozen in this step, but gradients still pass through it to guide the generator.

As training progresses, G produces more realistic images and the task of D becomes more difficult. At an ideal equilibrium, generated and real images follow the same distribution, so the discriminator cannot reliably distinguish them and outputs approximately 0.5. You can see the evolution of some generated images in the MNIST dataset in Figure 7.9.

Example: Learning to generate handwritten digits

Suppose that the training set contains handwritten digits. Initially, $G(z)$ looks like noise and D easily classifies it as fake. After repeated updates, G learns shapes, strokes, and eventually complete digits that resemble the training data. It is not asked to reproduce a particular digit; the latent vector determines which image is generated.

7.4.2 Instability and Mode Collapse

GAN training can be difficult because each network learns against an opponent that is changing at every iteration. The loss of one network therefore depends on the current state of the other network.

Common sources of instability include:

- **An overly strong discriminator.** If D separates real and fake images almost perfectly, it may provide weak gradients to G .
- **Unbalanced training.** If one network learns much faster than the other, the feedback exchanged between them may stop being useful.
- **Oscillation.** The two networks may repeatedly adapt to each other's latest behaviour instead of converging.
- **Loss interpretation.** A lower loss does not always imply better image quality, so generated samples and diversity must also be monitored.

A particularly important failure is **mode collapse**. It occurs when many different latent vectors produce the same image or only a small subset of the possible images:

$$z_1 \neq z_2 \quad \text{but} \quad G(z_1) \approx G(z_2).$$

For example, a GAN trained on all handwritten digits may generate realistic images of only the digit 3. The images look plausible, but the generator has failed to represent the diversity of the data distribution.

There is no single solution that guarantees stable GAN training. Common practical strategies include balancing the learning rates and update frequency of the two networks, using suitable normalisation, regularising the discriminator, and using alternative adversarial losses. Diversity should be evaluated together with visual quality so that mode collapse is not overlooked.

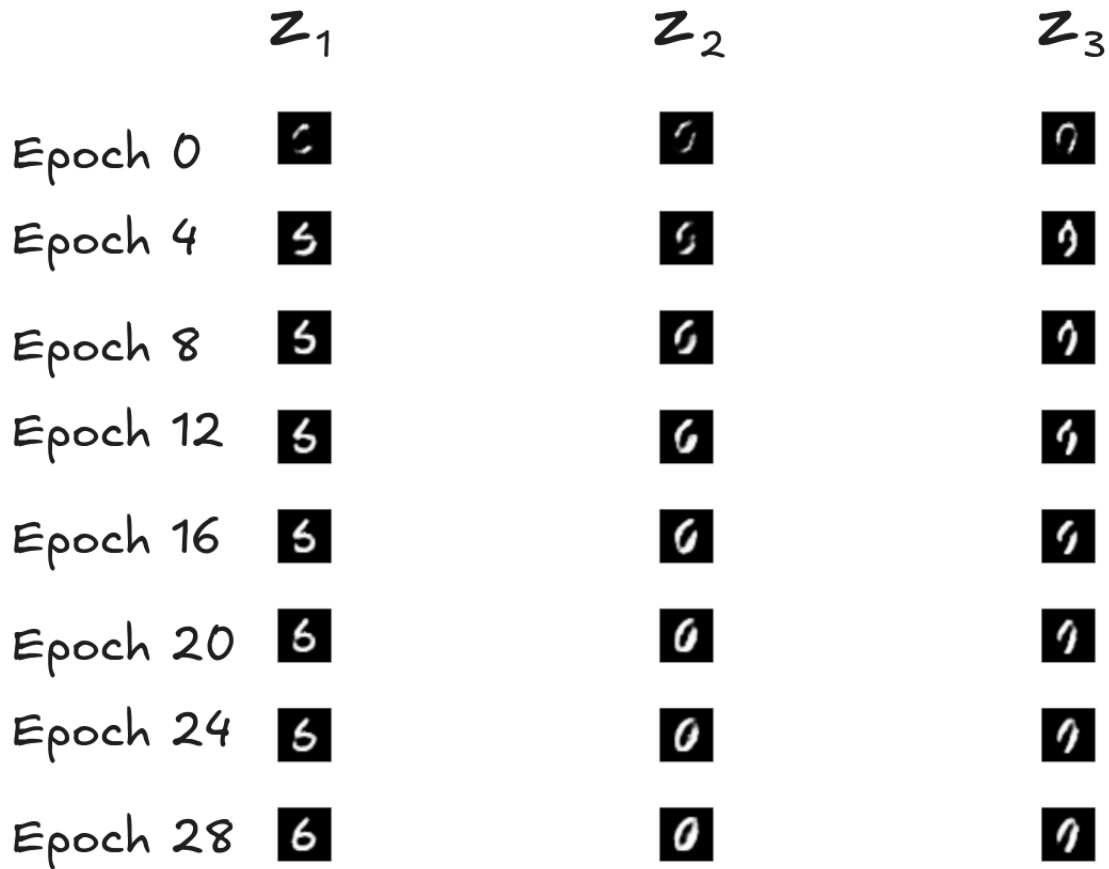


Figure 7.9: Evolution of $G(z)$ for some fixed latent vectors z_1 , z_2 and z_3 during the training.

7.4.3 Inference

After training, the discriminator is not needed to generate images. Inference uses only the generator:

1. Sample a latent vector $z \sim p_z$.
2. Compute the generated image $\hat{x} = G(z)$.

Each latent vector normally produces a complete image in one forward pass, so GAN inference is parallel and considerably faster than autoregressive PixelCNN sampling. Sampling multiple latent vectors produces multiple images.

Nearby latent vectors often generate visually related images. Consequently, interpolating between two vectors can produce a gradual transition between their generated images, although this behaviour is learned rather than guaranteed.

7.5 Conditional Generation

The models introduced before can generate new images, but they do not necessarily allow us to choose what appears in them. At most, we could sample latent vectors until the reconstructed image is similar to what we want, but that's inefficient. **Conditional generation** introduces additional information c and models

$$p(x \mid c)$$

instead of only $p(x)$. The condition may be a class label, an attribute, a text description, another image, or a spatial representation such as a segmentation mask.

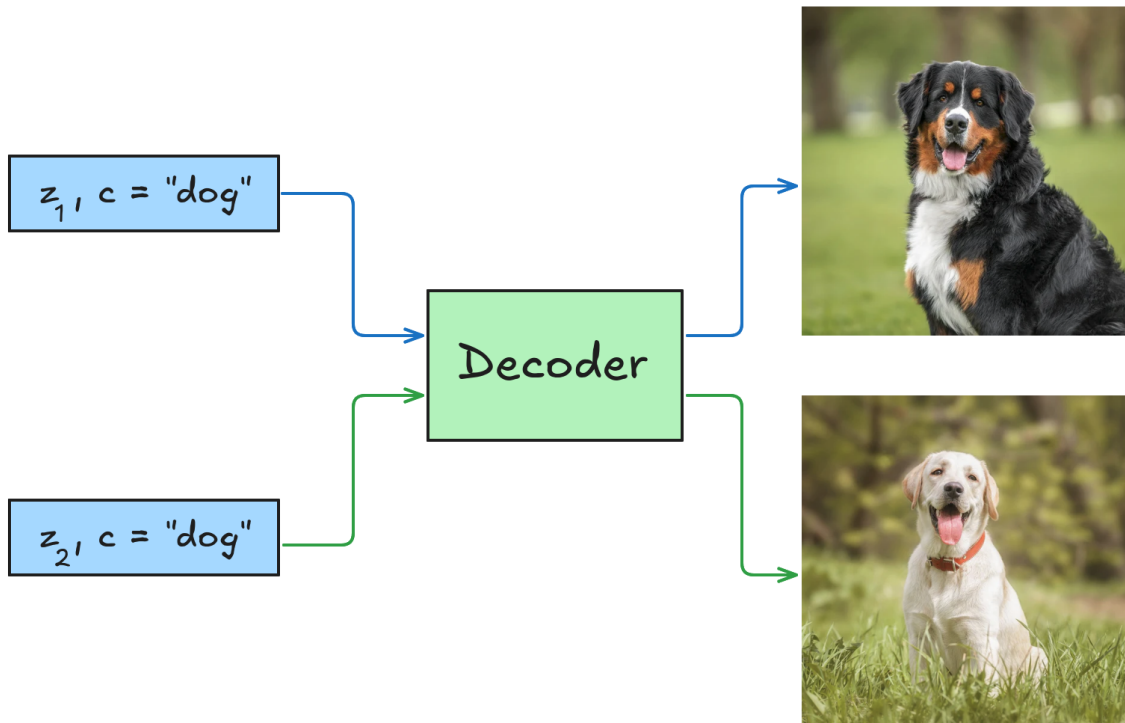


Figure 7.10: Given the same condition c , two different latent vectors z_1 and z_2 produce different outputs.

During training, the model receives examples (x_n, c_n) so that it can learn the relationship between each image and its condition. At inference time, we choose c and sample a latent vector z . The condition controls the requested content, while different values of z can produce different images compatible with that condition (Figure 7.10 shows an example):

$$\hat{x}_1 = F(z_1, c), \quad \hat{x}_2 = F(z_2, c).$$

We can pass the information c in two ways:

- If we want to include it to a one dimension vector, for example a latent vector z , we can concatenate the embedding representing c at the end of z .

- If we want to include it to an image, the standard way is to replicate the label as a “label map” (tensor). This means to 1) expand the one-hot label to match the spatial dimensions of the image, creating one additional channel per label class and 2) concatenate the label channels with the image channels along the channel dimension.

Example: Passing Information to an Image

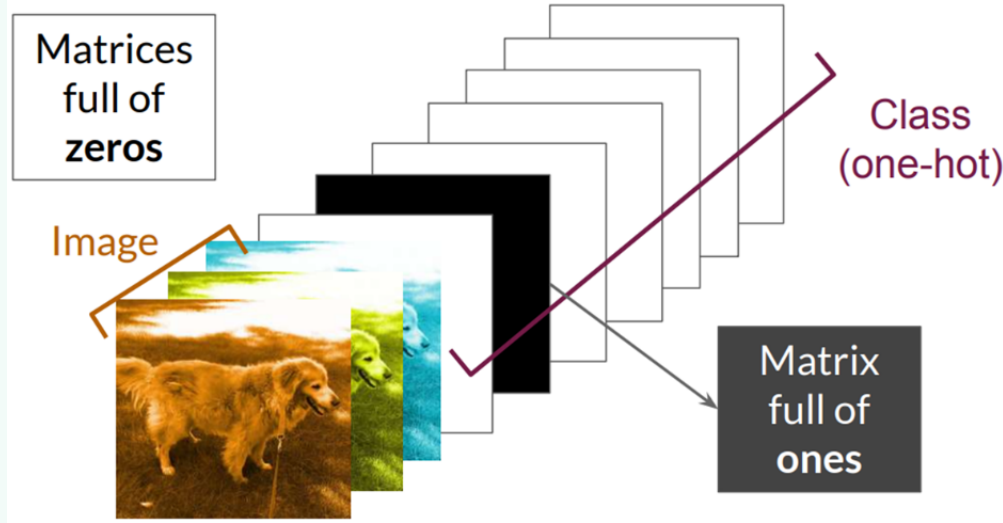


Figure 7.11: Including class information to an image.

If we want to condition an image $x \in \mathbb{R}^{C \times H \times W}$, and there are 10 classes, we can create a $10 \times H \times W$ label tensor, fill each channel with 1 or 0 (according to the one-hot label), and then concatenate this tensor and the original image to form a $(C + 10) \times H \times W$ tensor, as Figure 7.11 shows. Another way would be to use a trainable embedding.

7.5.1 Conditional VAE

In a **Conditional Variational AutoEncoder (CVAE)**, both the encoder and decoder receive the condition c . The encoder predicts a latent distribution conditioned on the input image and its condition:

$$q(z \mid x, c) = \mathcal{N}(z; \mu(x, c), \text{diag}(\sigma^2(x, c))) .$$

As in a standard VAE, the latent vector is sampled using

$$z = \mu(x, c) + \sigma(x, c) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

The decoder then uses both z and c to reconstruct the image:

$$\hat{x} = f(z, c).$$

The architecture is, very similar to the original VAE, is shown in Figure 7.12.

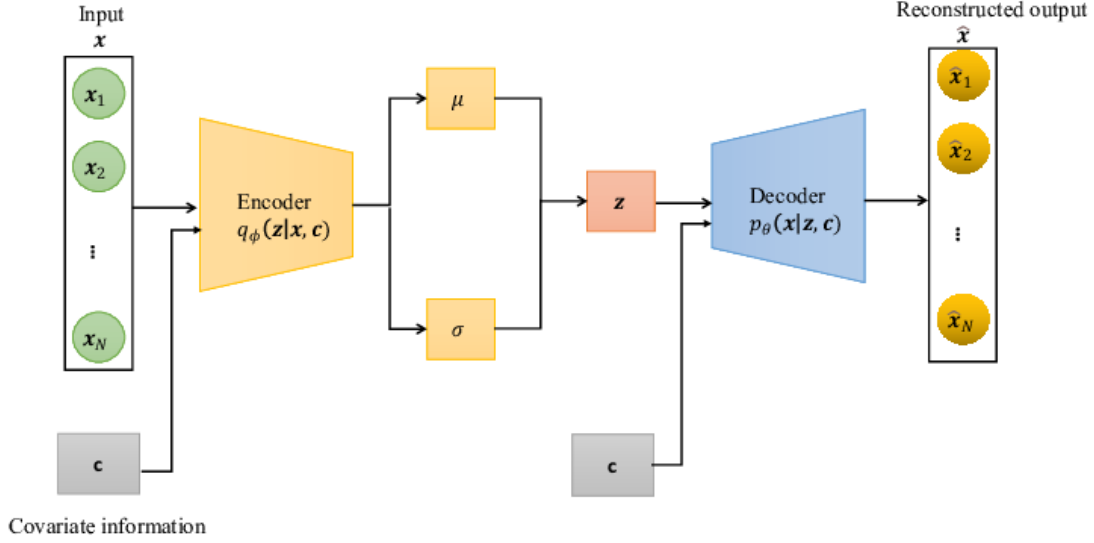


Figure 7.12: Conditional VAE.

The loss has the same two parts as the VAE loss:

$$\mathcal{L}_{\text{CVAE}} = \mathcal{L}_{\text{rec}}(x, \hat{x}) + D_{\text{KL}}(q(z | x, c) \| \mathcal{N}(0, I)).$$

The reconstruction term teaches the decoder to reproduce images compatible with c , while the KL term keeps the latent space suitable for sampling.

At inference time, the encoder is not required. We select a condition, sample $z \sim \mathcal{N}(0, I)$, and compute $\hat{x} = f(z, c)$. For example, if c represents the digit 7, different latent vectors can generate different handwritten sevens.

7.5.2 Conditional GAN

In a **Conditional GAN (cGAN)**, the condition is supplied to both the generator and the discriminator. The generator produces

$$\hat{x} = G(z, c),$$

and the discriminator evaluates an image together with its condition:

$$D(x, c) \in [0, 1].$$

Thus, the discriminator learns whether an image is real for the given condition, and the generator learns to create images that are compatible with that condition. The architecture is shown in Figure 7.13.

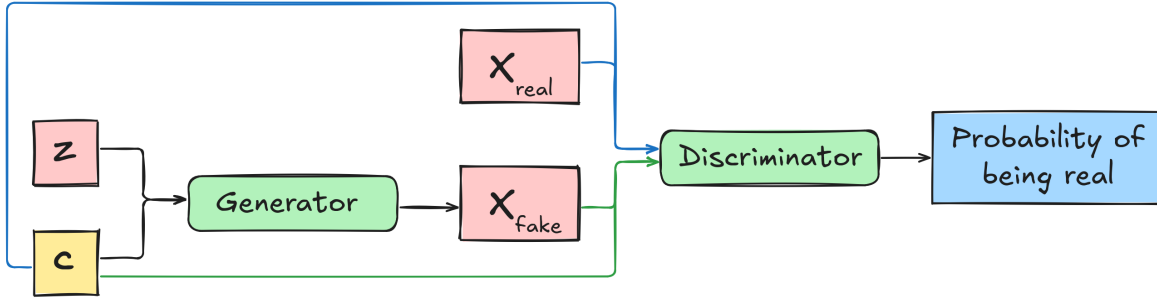


Figure 7.13: Conditional GAN.

For a batch of images—condition pairs (x_n, c_n) and latent samples z_n , the losses can be written as

$$\mathcal{L}_D = -\frac{1}{B} \sum_{n=1}^B [\log D(x_n, c_n) + \log (1 - D(G(z_n, c_n), c_n))],$$

and

$$\mathcal{L}_G = -\frac{1}{B} \sum_{n=1}^B \log (D(G(z_n, c_n), c_n)).$$

As in a standard GAN, the networks are trained alternately. The difference is that every real or generated image is now associated with a condition. At inference time, only G is needed: we choose c , sample z , and generate $G(z, c)$.

7.5.3 Controlled Generation: Beyond Labels

Class labels provide only global information. More detailed conditions can give the user greater control over the generated image:

- **Attributes** can specify properties such as colour, expression, or style. Several attributes may be active at the same time. We can use embeddings for this.
- **Text** can describe the desired objects, appearance, and scene. A text encoder first converts the description into a representation used by the generator.
- **Spatial conditions**, such as segmentation masks, edges, poses, or depth maps, specify where content should appear. Unlike a class label, they preserve spatial information.
- **Input images** can condition tasks such as colourisation, restoration, or image-to-image translation, using embeddings.

Note that we can use any of this information as a condition for either a CVAE or a cGAN. For example, instead of passing just the label, which may be a one-hot vector, we can pass a text embedding, a segmentation mask, or a reference image. The generator will then produce images that respect the condition, while the latent vector will control the remaining details. The condition constrains the result but does not normally define every pixel. This allows several plausible images to be generated from the same condition.

Example: Generating a street scene from a segmentation mask

Suppose that c is a segmentation mask containing regions for sky, road, buildings, and cars. A conditional generator uses this mask as a spatial guide. The generated image should respect the layout, but different latent vectors can produce different weather, colours, vehicle appearances, and building textures.

Warning: Desirable Properties of the Latent Space

There are some desirable properties we want to check in the latent space:

- **Continuity:** Small changes in z yield smooth, semantically gradual changes in the generated image.
- **Completeness:** Points in in the latent space decode to plausible data.
- **Semantic alignment:** Directions in Z correspond to interpretable attributes or factors.
- **Linear compositionality:** The concept that vector addition and subtraction in the latent space correspond to meaningful semantic operations in the real world. (see Figures 7.14 and 7.15).



Figure 7.14: Addition and subtraction in the latent space.



Figure 7.15: Interpolation in the latent space.

7.6 Exercises

Exercise 1: Autoregressive Factorisation

Factorise the joint distribution of a 2×2 grayscale image in raster order. State the context used to predict each pixel.

Exercise 2: Mask A and Mask B

Draw Mask A and Mask B for a 5×5 kernel in the grayscale case. Explain why the centre is excluded from Mask A but included in Mask B.

Exercise 3: Training and Generation

Explain why PixelCNN can predict all positions in parallel during training but must generate pixels sequentially at inference time.

Exercise 4: RGB Ordering

For one RGB pixel, write the autoregressive factorisation in the order $R \rightarrow G \rightarrow B$. Which current-pixel channels may be used when predicting red, green, and blue?

Exercise 5: Autoencoder Reconstruction

Describe the roles of the encoder, latent representation, and decoder in an autoencoder. Which value is used as the reconstruction target?

Exercise 6: AE and VAE Generation

Explain why sampling an arbitrary latent vector is unreliable for a standard AE, but meaningful for a trained VAE. Describe how a VAE generates a new image.

Exercise 7: AE and VAE Variants

Briefly explain the main idea of a denoising AE, sparse AE, β -VAE, and VQ-VAE. Give one situation in which each variant could be useful.

Exercise 8: GAN Training

Describe the roles of the generator and discriminator in a GAN. Explain why they are updated alternately and state the target used for real and generated images when training the discriminator.

Exercise 9: Mode Collapse

Explain mode collapse and give a simple example. Why is looking only at the realism of individual generated images insufficient to detect it?

Exercise 10: GAN Inference

Describe how a trained GAN generates a new image. Which network is needed at inference time, and how can a conditional GAN provide control over the output?

Exercise 11: Conditional Generation

Explain the difference between modelling $p(x)$ and $p(x \mid c)$. How can a conditional model generate different images for the same condition? Give one example of a useful condition other than a class label.